

Software Testing

The quality assurance that keeps your code working

Testing: Like Quality Control in Manufacturing

Real-world analogy: Car manufacturing

- Every car part is tested before assembly
- Final inspection before it leaves the factory
- Recall if problems are found later

In software:

- Every function/method should be tested
- Integration testing ensures parts work together
- Bug fixes or any changes trigger retesting
- Before shipping, check if all the requirements are met

Why Testing is Critical

No High-quality code without testing:

- Fixing bugs in production is cheap.
- You can refactor confidently.
- Saves debugging time.

- Serves as documentation.
- Enables CI/CD.
- Makes code more maintainable.
- Essential for career growth.
- Prevents embarrassing failures that damage user trust.

Testing is the most efficient way to make high-quality software

- **Prevents costly bugs** – Catch issues early
- **Builds confidence** – Refactor without fear
- **Saves time** – Pinpoints failures quickly
- **Documents behavior** – Shows how code should work
- **Enables deployment** – Safe, frequent releases
- **Reduces stress** – No 3 AM crashes

- **Improves maintainability** – Cleaner, modular code
- **Covers edge cases** – Finds hidden problems
- **Collaboration** – Shared understanding of behavior
- **Career Growth** – Professional standard
- **User Trust** – Prevents failures, protects reputation

Unit Tests: Testing the Building Blocks

- Testing light switches in a new house
 - Each switch should turn its specific light on/off
 - Test BEFORE connecting all the wiring
 - If one fails, fix it immediately

Test each function/method individually

Unit testing is a part of development, not testing.

Unit Testing Benefits

- **Early Bug Detection**
- **Safe Refactoring**
- **Living Documentation**
- **Design Feedback** as Hard-to-test code = bad design

Write tests as you develop, not after!

Types of Tests: The Testing Pyramid

- **Unit Tests** (Bottom - Most tests)
 - Test individual functions/methods
 - Fast and isolated
- **Integration Tests** (Middle)
 - Test how modules work together
 - Database + business logic
- **Acceptance Tests** (Top - Fewest tests)
 - Test complete user scenarios
 - Does the whole system work?

Regression Tests (Spanning All Levels)

- Can be unit, integration, or acceptance tests.
- Re-run after every code change, bug fix, or feature addition.
- Focus on previously tested features and known bug fixes.

Test Types Explained

- **Unit Test:** "Does this brick work?"
 - Tests one method/function at a time
- **Regression Test:** "Does the renovated room still work?"
 - Re-run tests after code changes

- **Integration Test:** "Do these rooms connect properly?"
 - Tests multiple modules together
- **Acceptance Test:** "Is the customer happy with the house?"
 - Tests complete user requirements

Unit Test Rules: The Golden Guidelines

1. Test Early, Test Often - Like checking ingredients while cooking
2. Make It Automatic - Set up continuous testing
3. Bug = Update Tests - Fix the bug AND prevent it from recurring